

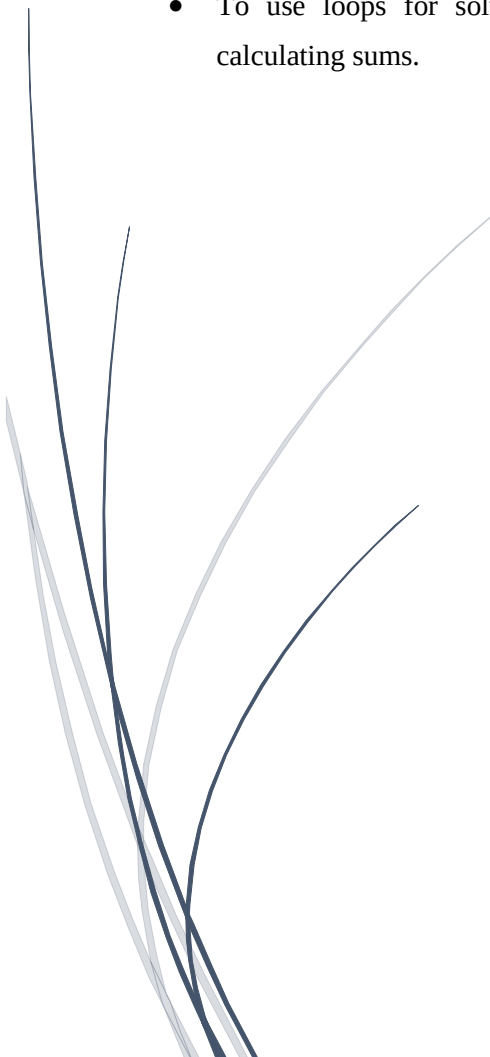
Lab 5

While loop in C++

Learning Objectives

The objective of this lab is to:

- To understand the working of the while loop in C++.
- To learn how to implement a do...while loop.
- To understand the concept of a nested while loop.
- To use loops for solving programming problems such as printing numbers and calculating sums.



Loop

Loops are used to repeat a block of code. Being able to have your program repeatedly execute a block of code is one of the most basic but useful tasks in programming -- many programs or websites that produce extremely complex output are really only executing a single task many times. Now, think about what this means: a loop lets you write a very simple statement to produce a significantly greater result simply by repetition.

While loop:

The most basic loop in C++ is the while loop. A while statement is like a repeating if statement. Like an If statement, if the test condition is true: the statements get executed. The difference is that after the statements have been executed, the test condition is checked again. If it is still true the statements get executed again. This cycle repeats until the test condition evaluates too false.

A while loop is a control flow statement that allows code to be executed repeatedly based on a given condition. The *while* consists of a block of code and a condition. The condition is first evaluated

and if the condition is true the code within the block is then executed. This repeats until the condition becomes false

Syntax:

```
While(expression){  
    statement 1;  
    statement 1;  
    inc, dec;  
}
```

The expression can be any combination of Boolean statements that are legal.

Example:

This program prints numbers from 1 to 5 using a while loop.

```
#include <iostream>
using namespace std;
int main() {
    int i = 1;           // initialize variable
    while (i <= 5) {     // condition
        cout << i << endl;
        i++;             // increment
    }
    return 0;
}
```

Nested Loop:

A nested loop in C++ is a loop placed inside another loop. The inner loop runs completely every time the outer loop runs once.

Syntax:

```
while(condition) { //Outer loop
```

```
    While{condition) { //inner loop
```

```
        // code
```

```
    }
```

In nested loops:

- the outer loop controls rows
- the inner loop controls columns

Nested loops are commonly used for:

- patterns
- matrices
- tables

Examples:

```
//Nested while loop  
#include <iostream>  
using namespace std;  
int main() {  
    int i = 1;  
    while(i <= 3) { //outer while loop  
        int j = 1;  
        while(j <= 2) { //Inner while loop  
            cout << "i = " << i << " , j = " << j << endl;  
            j++;  
        }  
        i++;  
    }  
    return 0;  
}
```

Do while Loop:

A do-while loop in C++ is a loop that executes the code block at least one time, and then checks the condition.

The condition is checked after executing the loop body.

Syntax:

```
#include <iostream>
using namespace std;
int main() {      //Main function
    int i = 1;
    do {          //starting the loop first
        cout << i << endl;
        i++;
    } while(i <= 5); //if the condition ture , loops starts again
    return 0;
}
```

Lab5: Lab Tasks

Task 1

1. Write a C++ program that uses a while loop to display numbers from 1 to 10
2. each on a separate line. with comments

```
#include <iostream>    // Header file for input and output
using namespace std;  // Standard namespace

int main() {
    int i = 1;         // Initialize variable i with 1
    while (i <= 10) {  // Loop runs until i is <= to 10
        cout << i << endl; // Display value of i on separate line
        i++;           // Increment value of i by 1
    }
    return 0;         // End of program
}
```

Task 2: Using Do while loop

Repeat the first task using a do-while loop.

```
#include <iostream>    // Header file for input and output
```

```
using namespace std;    // Standard namespace
```

```
int main() {  
  
    int i = 1;           // Initialize variable i with 1  
  
    do {  
  
        cout << i << endl; // Display value of i  
  
        i++;             // Increment value of i by 1  
  
    } while (i <= 10);    // Condition for loop  
  
    return 0;            // End of program  
}
```

Task 3: Sum Using while loop

Write a C++ program that:

1. Prompts the user to enter a positive integer.
2. Uses a while loop to calculate the sum from 1 to that number.
3. Displays the calculated sum.

```
#include <iostream>           // Header file for input and output

using namespace std;         // Standard namespace

int main() {

    int num;                  // Variable to store user input

    int i = 1;                // Initialize counter variable

    int sum = 0;              // Variable to store sum

    cout << "Enter a positive integer: ";

    cin >> num;               // Take input from user

    while (i <= num) {        // Loop runs from 1 to num

        sum = sum + i;        // Add current value of i to sum

        i++;                  // Increment i

    }

    cout << "Sum = " << sum << endl; // Display final sum

    return 0; }               // End of program
```